

Implementing a VSOP Compiler

Roland GREFFE, Pascal FONTAINE

1 Introduction

In this project, you will implement a compiler for the VSOP language. VSOP stands for ***Very Simple Object-oriented Programming*** language. It is **object-oriented**, **expression-based** and **statically typed**. All the required information about this language in order to implement your compiler is written in its **manual**, available on eCampus.

More precisely, you will implement the front-end part of a compiler, whose goal is to check the correctness of a VSOP source file and to generate the intermediate representation of this input program. As you will see during the theoretical courses, the front-end part of a compiler is composed of four steps:

1. the **Lexical analysis** part, where the source file (a stream of **characters**) is split into a stream of **tokens**;
2. the **Syntax analysis** part, where this stream is turned into a **syntax tree**;
3. the **Semantic analysis** part, which **annotates** the syntax tree;
4. the **Code generation** part, which generates the **intermediate representation** from the annotated syntax tree.

Once the intermediate representation is obtained, you will use an external compiler to generate the final executable.

This project can be done alone, by group of two (highly preferred option), or exceptionally by group of three (ask for permission, stating the reason). We **strongly recommend** that you do it in groups, because this is quite a large project.

2 Implementation

Concerning the language to implement the compiler, you can choose among **C/C++** or **Python**. We **encourage** you to use C++, as it provides high-level structures like classes, while still giving you a low-level control about what it does (unlike Python). It is also compatible with well-known lexer and parser generators that you can use in the two first parts, as well as a library to generate intermediate representation. However, for those who do not know this language (or those who really hate it), you can choose Python. There exist Python libraries mimicking the useful C/C++ libraries, but they maybe less stable and documented. Finally, if you want to use an other language, contact the assistant and we will discuss about that.

Note also that we will provide some examples about the compiler tools for C++, but we will not be able to provide in-depth support for all tools in the various programming languages.

Part	Deadline
Lexical analysis	25/02
Syntax analysis	18/03
Semantic analysis	15/04
Code generation	13/05

Table 1: Deadlines for the different parts of the project.

3 Submission

You will have **four deadlines** for this project, each one concerning one part of the compiler. The different deadlines are given in Table 1. They may change during the semester.

For each deadline you will receive a specific statement, telling you what you are supposed to do for that part. Each part will have its own project on the submission platform, and once your code is submitted, it will be tested with different input files (both correct and incorrect). **20% (5% for each part)** of your final grade will be determined **by these automated tests**.

For the last deadline, you will also have to write a **short report** describing your compiler, the algorithms you have used, the difficulties you have faced, etc. We will also ask you to estimate the time spent on this project, therefore you can already start tracking it.

4 Docker container

All the tests will be run on a **Docker container**, to which you have access. This allows you to test locally your code on the same environment than the one of the submission platform.

To use it, first install docker on your computer. Once this is done, you can obtain the docker image with:

```
> docker pull cffs/compilers
```

Note: on Linux, to execute docker commands with your user, you might have to add your user to the docker group

```
> usermod -aG docker
```

and you might have to logout and in again.

Once downloaded, a container can be started from the image with:

```
> docker run --rm -it cffs/compilers /bin/sh
```

This basically starts the container and opens a shell (-it option and /bin/sh argument) in the container. The option --rm tells Docker to remove the container when it will be stopped. You are connected by default as root. It is usually not a good idea to stay root in a terminal, thus you can switch to the other user, vagrant, whose password is also vagrant:

```
> su vagrant
```

You can then reach its home directory (/home/vagrant) with:

```
> cd /home/vagrant
```

When you are done with the container, you can exit the shell or close your terminal, this will stop the container and remove it (thanks to the -rm option).

The container has its own filesystem, which means that it does not have access to your files. You can use `docker cp` to copy files to the container, but a faster solution is to link a directory of yours to a directory of the container. This can be done when starting the container:

```
> docker run --rm -it -v "$(pwd):/home/vagrant/compilers" cffs/compilers /bin/sh
```

This links your current working directory (obtained with the `pwd` command) to the directory of the container named `/home/vagrant/compilers`. Note that any modification made by the container in this directory will impact your own directory.

For Windows users, the `pwd` may not work, you can simply replace it by the path to the folder you would like to link to the docker.

As this environment is the same as the one on which are tested your codes on the submission platform, we **strongly** encourage you to test locally your code with this container.

5 Questions

We will organize Q&A sessions throughout the semester, usually after the theoretical course. There are also several forums on eCampus, one for each part of the project. The questions asked during the previous years are still there. If you have any problem, first **have a look** to these forums, as the students often encounter the same difficulties each year. Otherwise, add a new question to the forum. **Do not post code** on the forum, so if you must share some code with us, you can send the assistant an email. Here is his email:

- r.greffe@uliege.be

When sending an email (or when asking a question on the forum), be sure to give all the necessary information to understand your problem and be as precise as possible (including error messages, input files, what you expected, etc.).